

TUTORIAL

Criando o Jogo Termo com HTML, CSS e JavaScript

O que vamos construir?

Neste tutorial você vai criar um clone do jogo **Termo** (versão brasileira do Wordle) usando apenas HTML, CSS e JavaScript puro — sem nenhuma biblioteca ou framework externo. O projeto é dividido em três arquivos: **index.html**, **styles.css** e **App.js**.

Ao final você terá um jogo completamente funcional onde o jogador tem 6 tentativas para adivinhar uma palavra de 5 letras. Cada letra revelada é colorida em **verde** (posição correta), **amarelo** (letra existe, posição errada) ou **cinza** (letra não existe na palavra).

Pré-requisitos

Você precisa apenas de um editor de texto (VS Code, Notepad++, etc.) e um navegador moderno (Chrome, Firefox, Edge). Nenhuma instalação adicional é necessária.

Passo 1 — Estrutura de pastas

Crie uma pasta chamada **termo/** e, dentro dela, os três arquivos:

```
termo/  
  index.html  
  styles.css  
  App.js
```

Essa separação segue a boa prática de manter estrutura (HTML), visual (CSS) e comportamento (JS) em arquivos independentes, tornando o código mais fácil de ler e manter.

Passo 2 — Criando o index.html

O arquivo HTML define a estrutura da página. Ele possui quatro elementos principais: o cabeçalho com o nome do jogo, o tabuleiro de tentativas (**#board**), a área de mensagens (**#message**) e o teclado virtual (**#keyboard**).

Código do index.html:

```
<!DOCTYPE html> <html lang="pt-BR"> <head> <meta charset="UTF-8" /> <link  
rel="stylesheet" href="styles.css" /> <title>Termo</title> </head> <body> <header>  
<h1>TERMO</h1> <p class="subtitulo">Adivinhe a palavra em 6 tentativas</p> </header>  
<main> <div id="board"></div> <div id="message"></div> <div id="keyboard"></div>  
</main> <script src="App.js"></script> </body> </html>
```

Importante: O `<script>` é carregado no final do `<body>` para garantir que todos os elementos do DOM já existam quando o JavaScript tentar acessá-los.

Passo 3 — Estilizando com styles.css

O CSS define a aparência escura característica do jogo, as células do tabuleiro e as animações de reveal, shake e bounce.

3.1 — Fundo escuro e centralização

```
body { background: #121213; color: #fff; display: flex; flex-direction: column; align-items: center; min-height: 100vh; }
```

Usamos **Flexbox** no body para centralizar todo o conteúdo horizontalmente. **min-height: 100vh** garante que a página ocupe pelo menos toda a altura da tela.

3.2 — Células do tabuleiro (.tile)

```
.tile { width: 62px; height: 62px; border: 2px solid #3a3a3c; font-size: 2rem; font-weight: 700; display: flex; align-items: center; justify-content: center; }
```

3.3 — Estados de cor após revelação

Classe CSS	Cor	Significado
.tile.correct	Verde #538d4e	Letra na posicao certa
.tile.present	Amarelo #b59f3b	Letra existe, posicao errada
.tile.absent	Cinza #3a3a3c	Letra nao existe na palavra

3.4 — Animacoes CSS

Tres animacoes dao vida ao jogo:

- **flip** — rotaciona a celula no eixo X ao revelar o resultado;
- **shake** — faz a linha tremer quando a palavra e invalida;
- **bounce** — cada tile da linha vencedora pula em sequencia.

```
@keyframes flip { 0% { transform: rotateX(0deg); } 50% { transform: rotateX(-90deg); } 100% { transform: rotateX(0deg); } }
```

Passo 4 — A logica do jogo em App.js

O JavaScript e o coracao do projeto. Esta dividido em seis secoes bem delimitadas.

1. Banco de palavras

Array BANCO com palavras validas de 5 letras. A palavra secreta e sorteada aleatoriamente com Math.random().

2. Estado do jogo

Seis variaveis controlam tudo: palavraSecreta, tentativas, tentativaAtual, jogoAtivo, linhaAtual e as constantes MAX_TENTATIVAS e TAMANHO.

3. Construção da interface

buildBoard() cria 6x5 divs no DOM. buildKeyboard() cria os botoes do teclado virtual. Ambas sao chamadas a cada novo jogo.

4. Logica de input

handleKey() processa teclas fisicas e virtuais. adicionarLetra() insere no tile. removerLetra() apaga. Um listener de keydown conecta o teclado fisico.

5. Verificacao da tentativa

submeterTentativa() valida o tamanho, checa o banco, calcula o resultado com calcularResultado() e dispara revelarLinha() e atualizarTeclado().

6. Funcoes auxiliares

showMessage(), shakeRow(), bounceRow() e encerrarJogo() cuidam do feedback visual para o jogador.

Passo 5 — O algoritmo de coloracao

Esta e a parte mais importante da logica. A funcao **calcularResultado()** percorre a tentativa em duas passagens para evitar colorir letras duplicadas incorretamente:

```
function calcularResultado(tentativa, secreta) { const resultado =
Array(5).fill('absent'); const contagem = {}; // 1a passagem: marca as letras corretas
for (let i = 0; i < 5; i++) { if (tentativa[i] === secreta[i]) { resultado[i] =
'correct'; } else { contagem[secreta[i]] = (contagem[secreta[i]] || 0) + 1; } } // 2a
passagem: marca letras presentes na posicao errada for (let i = 0; i < 5; i++) { if
(resultado[i] !== 'correct' && contagem[tentativa[i]] > 0) { resultado[i] = 'present';
contagem[tentativa[i]]--; } } return resultado; }
```

Por que duas passagens? Se a palavra secreta é 'CARRO' e o jogador digita 'RARCO', o primeiro R não pode ser marcado como amarelo se o segundo R já ocupou a posição correta. A primeira passagem reserva as posições corretas antes de avaliar as restantes.

Passo 6 — Revelação animada das células

A função **revelarLinha()** usa **setTimeout** com delays escalonados (300ms por coluna) para criar o efeito de revelação progressiva, como no jogo original.

```
function revelarLinha(row, tentativa, resultado) { for (let c = 0; c < 5; c++) { const
tile = document.getElementById('tile-'+row+'-'+c); setTimeout(() => {
tile.classList.add('flip'); // inicia animação setTimeout(() => {
tile.classList.add(resultado[c]); // aplica cor no meio }, 250); }, c * 300); // delay
por coluna } }
```

Passo 7 — Inicialização e novo jogo

A função **iniciarJogo()** é chamada automaticamente ao carregar a página e também pelo botão 'Novo Jogo'. Ela sorteia uma nova palavra e reconstrói toda a interface do zero:

```
function iniciarJogo() { palavraSecreta = BANCO[Math.floor(Math.random() *
BANCO.length)]; tentativas = []; tentativaAtual = ''; jogoAtivo = true; linhaAtual = 0;
buildBoard(); buildKeyboard(); } // Chama ao carregar a página: iniciarJogo();
```

Dicas para expandir o projeto

- **Mais palavras:** Expanda o array BANCO com centenas de palavras para aumentar a variedade. Você pode importar um arquivo JSON com `fetch()`.
- **Palavra do dia:** Use a data atual como semente para `Math.random()` e assim todos os jogadores recebem a mesma palavra no mesmo dia.
- **Contador de tentativas:** Adicione um placar que registra quantas vezes o jogador venceu em 1, 2, 3... tentativas, usando `localStorage`.
- **Modo difícil:** Implemente uma regra onde letras verdes e amarelas reveladas devem ser usadas nas tentativas seguintes.
- **Animação de compartilhamento:** Gere automaticamente o grid de emojis (verde/amarelo/cinza) para o jogador colar em redes sociais após o jogo.

Parabéns! Você agora tem um jogo Termo completo e funcional construído do zero.